EE2115 Lab2 Pre-lab

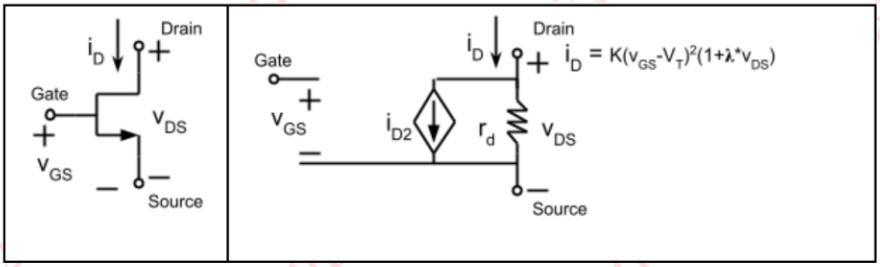


Figure 1: Active-Region Model.

Given the active region model of the MOSFET in Figure 1, let's first assume that $\lambda = 0$, meaning the r_d resistor is not present. Also, we assume the small-signal approximation, so the quadratic term is small enough to ignore. Figure 2 shows a standard simplified small-signal model where $I_D = g_m V_{GS}$. This makes it clear that when $V_{GS} = 0$, the dependent current source acts like an open circuit.

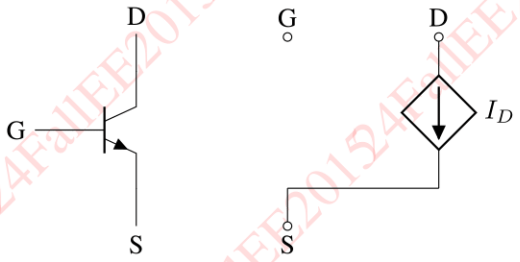


Figure 2: Small Signal Model. $I_D = g_m V_{GS}$.

Therefore, we can easily redraw the equilibrium circuit from the first circuit in Lab 2, Week 2, as shown in Figure 3. Perform nodal analysis at node A to find $V_A(t)$ for this new circuit when $V_{GS} = 0$ and $V_{GS} = C$, where C is some constant.

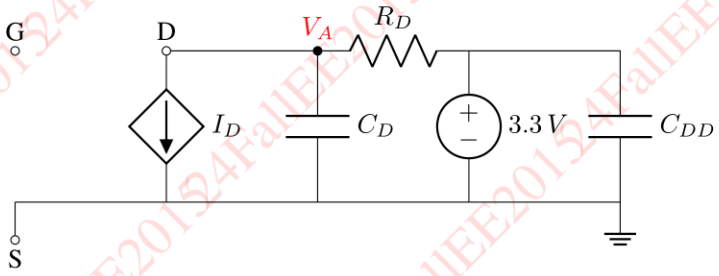


Figure 3: Equilibrium Circuit.

Now, using nodal analysis, we have:

$$\begin{aligned} I_D + C_D \frac{d\,V_A}{d\,t} + \frac{V_A - 3.3\,V}{R_D} &= 0 \\ \Rightarrow R_D\,C_D \frac{d\,V_A}{d\,t} + V_A &= 3.3 - R_D\,I_D, \end{aligned} \tag{1}$$

which is a first-order differential equation with the initial condition $V_A(0) = v_0$. One simple way to check if equation (1) is correct is by looking at the **units**. We have voltage on both sides of the equation. Therefore, it is important to ensure the correct units in every step of the derivation.

Solving IVP

Now, how can we find $V_A(t)$ from this initial value problem (IVP)? You can use one of the following methods:

1. Guess the solution (it should be in exponential form).
2. Use the Laplace transform.
3. Separation of variables

However, you might wonder: if I guess an exponential form, is it the only possible solution? In other words, **what if** there is another function $V_A(t)$ that also satisfies all conditions but is different from my guess? The short answer is that the solution is unique, but proving uniqueness requires more advanced math techniques. Instead of proving it, I will show another way to solve it—separation of variables.
In general, we want to find $v(t)$ that satisfies the following conditions:

$$\alpha \frac{d\,v}{d\,t} + v = c, \text{ with } v(0) = v_0. \tag{2}$$

First, we separate the variables by moving all terms with v to one side and all terms with t to the other side:

$$\begin{aligned} &\alpha \frac{d\,v}{d\,t} + v = c \\ \Rightarrow &\alpha\, d\,v = c\, d\,t - v\, d\,t \\ \Rightarrow &\alpha \frac{1}{c - v}\, d\,v = d\,t. \end{aligned} \tag{3}$$

Now, we change the variable by defining:

$$u = c - v \Rightarrow d\,u = -d\,v. \tag{4}$$

Next, we substitute it into the equation and then integrate both sides:

$$\begin{aligned} &-\alpha \frac{1}{u}\, d\,u = d\,t \Rightarrow -\alpha \int \frac{1}{u}\, d\,u = \int dt \\ \Rightarrow &-\alpha\, \ln(|u|) = t + c_0 \\ \Rightarrow &|u| = \exp\left(-\frac{t + c_0}{\alpha}\right) \\ \Rightarrow &|c - v(t)| = \exp\left(-\frac{t + c_0}{\alpha}\right) \\ \Rightarrow &v(t) = (v_0 - c) \cdot \exp\left(-\alpha^{-1}\, t\right) + c. \text{ consider the initial values.} \end{aligned} \tag{5}$$

Now, comparing with our original equation (1), we get:

$$\alpha = R_D\, C_D, \quad c = 3.3 - R_D\, I_D$$

$$V_A(t) = (V_0 - 3.3 + R_D\, I_D) \cdot \exp\left(-\frac{t}{R_D C_D}\right) + 3.3 - R_D\, I_D, \tag{6}$$

where $I_D = g_m\, V_{GS}$. You can check if this satisfies all the conditions in the equation (1) as well. It is important to note that $3.3\,V - R_D\, I_D$ **cannot be lower** than V_{DS} for the transistor to remain in the active region (i.e., saturation region). Otherwise, we need to use a different model for non-saturated V_{DS}. Figure 4 shows the simulation result.

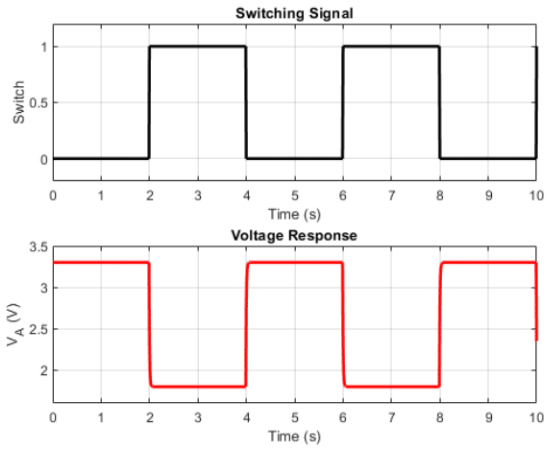


Figure 4: Simulation Result shows that the circuit behaves like a "NOT Gate".

[Appendix] Algorithmic perspective for the uniqueness of the solution.

This part is optional and just for fun. Even many lecturers or instructors who are not very skilled may not fully understand it, but they can still do their job. The convergence of the initial value problem (IVP) is explained in a more algorithmic way. Given an IVP (7), we can compute an approximate solution step by step using Algorithm 1. It is provable that if the problem meets certain conditions, then as $K \to \infty$, it will converge to a unique solution.

$$\dot{v} = f(v), \text{ Initial Point } v(0) \text{ is given.} \tag{7}$$

Algorithm 1 Picard Iteration to solve IVP (7).

1: **Input:** Initial Guess (from the initial value) $v^0(t)$.
2: **for** $k = 1, 2, \ldots, K$ **do**
3: $\quad v^{k+1}(t) = v(0) + \int_0^t f\left(v^k(s)\right)\, ds$.
4: **end for**

Consider the simple initial value problem (IVP) (8) with an initial guess for the solution: $v^0(t) = 1$.

$$\dot{v} = -v, \; v(0) = 1. \tag{8}$$

The first few iterations are as follows:

$$\textbf{k=1: } v^1(t) = v_0 + \int_0^t -v^0(s)\, ds = 1 - t$$

$$\textbf{k=2: } v^2(t) = v_0 + \int_0^t -v^1(s)\, ds = 1 + \int_0^t -(1-s)\, ds = 1 - t + \frac{1}{2}t^2$$

$$\textbf{k=3: } v^3(t) = v_0 + \int_0^t -v^2(s)\, ds = 1 + \int_0^t -\left(1 - s + \frac{1}{2}s^2\right) ds = 1 - t + \frac{1}{2}t^2 - \frac{1}{6}t^3$$

$$\textbf{k=K: } v^K(t) = \sum_{k=0}^{K} (-1)^k \frac{t^k}{k!}$$

Therefore, we can conclude that the solution follows an exponential decay:

$$v(t) = \lim_{K \to \infty} v^K(t) = \sum_{k=0}^{\infty} (-1)^k \frac{t^k}{k!} = \exp(-t). \tag{9}$$

EE2115 Lab3 Pre-lab

Figure 1 shows the active region model in your Lab 3 Week 2 circuit. In Lab 2 Week 2, we treated I_D as a constant current source and did not consider that it actually depends on the input voltage $\mathbf{v_{GS}}$.

In reality, when designing a circuit, we need to account for this relationship. However, this can be challenging because the relationship is nonlinear, making analysis more difficult.

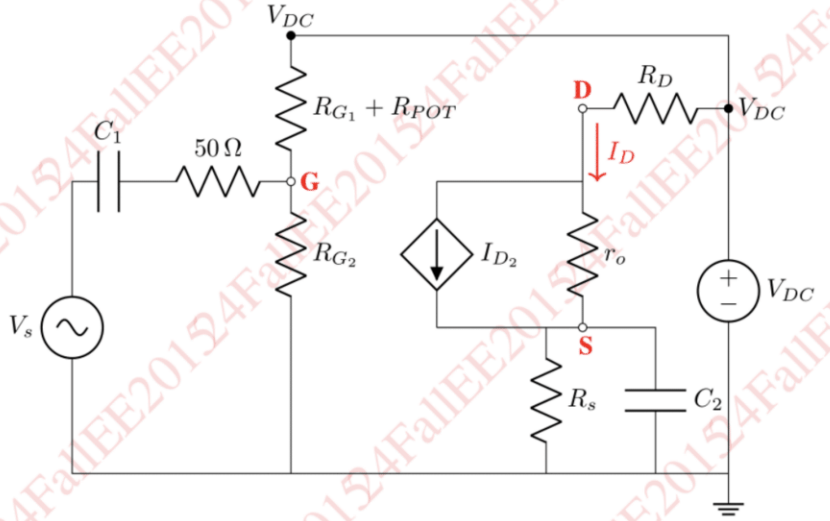


Figure 1: Equivalent Circuit for Analyzing the Active Region.

Fortunately, we can use a Taylor series expansion to approximate the function as a linear expression around a specific operating point. In other words, consider a nonlinear function I_D that depends on v_{GS}:

$$I_D(\mathbf{v_{GS}}) = K(\mathbf{v_{GS}} - V_T)^2(1 + \lambda \cdot V_{DS}), \tag{1}$$

where $\mathbf{v_{GS}}$ is a variable, and K, V_T, λ, and V_{DS} are fixed values.

Now, let's consider an arbitrary DC voltage $v^\star_{GS}$ as the operating point and perform a Taylor series expansion around this point. That is:

$$\begin{aligned} I_D(\mathbf{v_{GS}}) &= I_D(v^\star_{GS}) + \left.\frac{d\, I_D(\mathbf{v_{GS}})}{d\, \mathbf{v_{GS}}}\right|_{v^\star_{GS}} \cdot (\mathbf{v_{GS}} - v^\star_{GS}) + \frac{1}{2!}\left.\frac{d^2\, I_D(\mathbf{v_{GS}})}{d^2\, \mathbf{v_{GS}}}\right|_{v^\star_{GS}} \cdot (\mathbf{v_{GS}} - v^\star_{GS})^2 + \dots \\ &= I_D(v^\star_{GS}) + \underbrace{2\,K\,(v^\star_{GS} - V_T)\,(1 + \lambda \cdot V_{DS})}_{g_m(v^\star_{GS})}(\mathbf{v_{GS}} - v^\star_{GS}) + 2\,K\,(1 + \lambda \cdot V_{DS})(\mathbf{v_{GS}} - v^\star_{GS})^2 \\ &:= \text{DC Current} + g_m(v^\star_{GS}) \cdot \text{AC Amplitude} + 2\,K\,(1 + \lambda \cdot V_{DS}) \cdot (\text{AC Amplitude})^2 \end{aligned} \tag{2}$$

If the AC amplitude is small, we can assume that the quadratic term is approximately zero. This approach is generally called **small perturbation analysis**.

Since the circuit is approximately linear around the operating point, we can use **superposition** from linear circuit analysis. This means we can further break everything down into a DC part and an AC part. Because we are only interested in the AC component's behavior, superposition tells us that we can turn off all DC sources. In other words, all DC voltage sources are treated as short circuits. This leads to common-source small-signal analysis, allowing us to further simplify our model in Figure 1.

In common-source small-signal analysis, we focus only on how changes in the input signal (the AC component) affect the output. This means we can treat all DC sources as if they are short circuits because they only provide a constant voltage (DC offset) and do not influence the AC behavior. We call this an "AC ground." Additionally, capacitors in the circuit are used to block DC and allow AC to pass, so we assume they act like short circuits in AC

analysis. As a result, in the circuit from Week 2, when using the small-signal model for AC analysis, the source is effectively connected to ground. Figure 2 shows the small-signal models used for AC analysis.

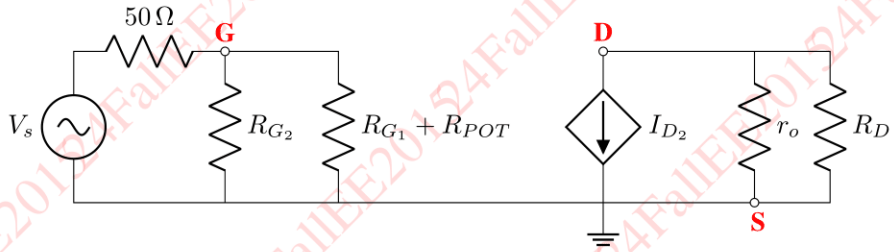


Figure 2: Equivalent Circuit for AC analysis.

Here, the MOSFET has an infinitely input impedance. Now, you can use nodal analysis to calculate the gain, given that the drain current is $I_{D_2} = g_m(v^\star_{GS}) \cdot V_{GS}$ and the output impedance is $Z_{out} = r_o \parallel R_D$. (This depends on how you define I_D. According to the definition in your lab manual, Z_{out} should be only R_D.)

Here, it is important to emphasize that the source is an **AC ground**, not a **DC ground**. This means that the AC component of V_S is zero, but the DC component is not.

Another question might be: can we analyze both the DC and AC components together instead of only the AC part? The answer is that this would lead back to nonlinear analysis. Only when the amplitude of the AC component is small around the operating point can we locally treat the circuit as linear. However, the overall circuit remains nonlinear.

[Appendix] Transfer Function

I'm adding this here even though it's not directly related to the lab for now because the final problem on your EE 2015 final exam was the simplest one, yet no one managed to get it completely right. I had to regrade it multiple times just to keep the average reasonable. I even wrote and sent a solution for a past exam a week before the final!

Consider the first circuit from Lab 2, Week 2:

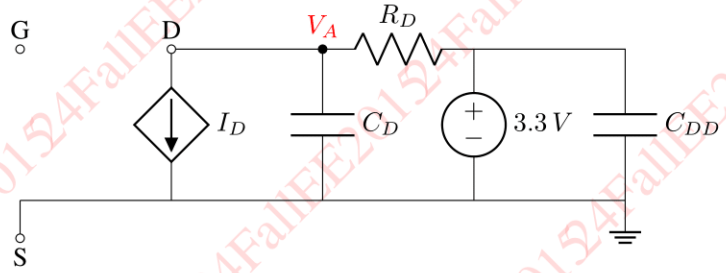


Figure 3: Circuit from lab 2.

Now, using nodal analysis, we have:

$$\begin{aligned} &I_D + C_D \frac{dV_A}{dt} + \frac{V_A - 3.3\,V}{R_D} = 0 \\ \Rightarrow &R_D\,C_D \frac{dV_A}{dt} + V_A = 3.3 - R_D\,I_D, \end{aligned} \tag{3}$$

Let's take the Laplace transform using $R_d C_d := \alpha$, with $3.3 - R_D I_A := v_i(t)$ and the initial condition $v(0) = 0$ (**Important!** You can always shift the initial time so that $v(t_0) = 0$. Even a DC input is 0 before you turn it on.).

$$\begin{aligned} &\alpha\, \dot{v}(t) + v(t) = v_i(t) \\ \Rightarrow &\alpha\, s\, V(s) - v(0) + V(s) = V_i\,(s) \\ \Rightarrow &T(s) = \frac{V(s)}{V_i(s)} = \frac{1}{\alpha\, s + 1} = \frac{1}{R_d\, C_d\, s + 1} \end{aligned}, \tag{4}$$

which means the transfer function depends entirely on your circuit, without considering any input voltage.

Now, let's see what the output looks like for a given frequency ω_0. First, we know that the Laplace transform of $\sin(\omega_0\, t)$ is:

$$V_i(s) = \mathcal{L}(\sin(\omega_0\, t)) = \frac{\omega_0}{s^2 + \omega_0^2}. \tag{5}$$

Thus, the output voltage is:

$$V(s) = T(s) \cdot V_i(s) = \frac{1}{\alpha\, s + 1} \cdot \frac{\omega_0}{s^2 + \omega_0^2}. \tag{6}$$

It may look complicated, but if you take the inverse Laplace transform, you will get:

$$v(t) = |T(j\,\omega_0)| \cdot \sin(\omega_0\, t + \angle T(j\,\omega_0)). \tag{7}$$

Thus, for an input consisting of two sine waves with frequencies ω_0 and ω_1, and any arbitrary transfer function $T(s)$:

$$v_i(t) = A_0 \cdot \sin(\omega_0\, t) + A_1 \cdot \sin(\omega_1\, t). \tag{8}$$

The output voltage is:

$$\begin{aligned} v(t) = &|\mathbf{T(s = j\,\omega_0)}| \cdot A_0 \cdot \sin(\omega_0\, t + \angle \mathbf{T(j\,\omega_0)}) \\ &+ |\mathbf{T(s = j\,\omega_1)}| \cdot A_1 \cdot \sin(\omega_1\, t + \angle \mathbf{T(j\,\omega_1)}). \end{aligned} \tag{9}$$

You should be able to find these on your Bode plot because when plotting a Bode plot, we set $s = \sigma + j\omega$ and consider $\sigma = 0$, focusing only on how ω changes.

Also, the problem in your EE 2015 final was simple because you just needed to **find the input sine wave frequency, plug it into your transfer function, and you would get everything**!

Another important point is that you can think of your circuit as generating an internal signal, and you perform a frequency-wise product with your input signal.

In other words, for each fixed s (frequency), you take the dot product. This frequency-wise dot product in the frequency domain corresponds to convolution in the time domain, and vice versa.

That means if you have an internal circuit signal and an input signal in the time domain, you can flip one of them, shift it, and take the product sum—just like a sliding-window moving average, which is a special case of this process.

EE2115 Lab3: Linear Regression

In this tutorial, I'll give a quick introduction to Linear Regression, also known as Mean Square Error (MSE) Minimization or Least Squares. Rather than starting with complex math, let's begin with a simple example from EE2015. Here's a table of made-up data, assuming we measured the following values:

	Data 1	Data 2	Data 3
Temperature (°F)	32	80	100
Voltage (mV)	-8.	0.2	0.3

Table 1: Fake Measurement.

The goal is to find a line such that:

$$\underbrace{\begin{bmatrix}-8.\\0.2\\0.3\end{bmatrix}}_{y} = \underbrace{\begin{bmatrix}32 & 1\\80 & 1\\100 & 1\end{bmatrix}}_{X}\underbrace{\begin{bmatrix}m\\b\end{bmatrix}}_{\theta}, \tag{1}$$

where m represents the slope and b is the offset (bias). However, since the matrix X is singular, we can only use projection:

$$X^\top (y) = X^\top (X\theta) \Rightarrow \theta = \left(X^\top X\right)^{-1} X^\top y. \tag{2}$$

This provides a closed-form solution for θ.

```
X = [32 1; 80 1; 100 1];
y = [-8.; 0.2; 0.3];
theta = inv(X'*X)*X'*y
>> theta = [0.1310; -11.7576]
```

Unfortunately, the closed-form solution has a computational complexity of $O(n^3)$ due to the matrix inversion, where n is the number of data points. For large n, this approach becomes slow. Therefore, in practice, gradient descent is often used as an alternative to the closed-form solution (e.g., using `LinearRegression` from `sklearn.linear_model` in Python).

Why is it also called least squares?

Linear regression is also known as mean square error minimization or least squares because it offers a different perspective in the realm of mathematical optimization.

To explain this in more abstract terms using the same example: Consider n data points, each with m features (in the table from the example, $m = 1$ because it only includes one feature called Temperature. However, we could have multiple features like Temperature and Moisture, etc.). Therefore, the matrix $X \in \mathbb{R}^{n\times(m+1)}$, vector $y \in \mathbb{R}^n$, and vector $\theta \in \mathbb{R}^{m+1}$ take the following form:

$$X = \begin{bmatrix}x_1\\x_2\\\vdots\\x_n\end{bmatrix},\ y = \begin{bmatrix}y_1\\y_2\\\vdots\\y_n\end{bmatrix},\ \theta = \begin{bmatrix}\theta_1\\\theta_2\\\vdots\\\theta_{m+1}\end{bmatrix}, \tag{3}$$

where x_i is a vector representing each data point with $m+1$ entries, which includes the number of features plus a bias term. y_i represents the output values for each data point, and θ_j is the coefficient for feature j that determines its influence in the linear model.

Now, we can calculate the error for each data point, add them all together, and then find the average:

$$\begin{aligned}\text{error}_i &= (y_i - x_i\theta)^2\\ \text{MSE} &= \frac{1}{n}\sum_{i=1}^{n}\text{error}_i = \frac{1}{n}(y - X\theta)^\top (y - X\theta).\end{aligned} \tag{4}$$

We aim to find the best value for θ that minimizes the mean square error (MSE). From basic calculus, we know that to do this, we take the derivative (gradient) with respect to θ and set it to 0:

$$\frac{\partial \text{ MSE}}{\partial \theta} = X^\top X\theta - X^\top y = 0 \Rightarrow \theta = \left(X^\top X\right)^{-1} Xy. \tag{5}$$

Therefore, linear regression aims to minimize the mean square error. Figure 1 shows the fitting results based on the data in Table 1 and the errors for each data point.

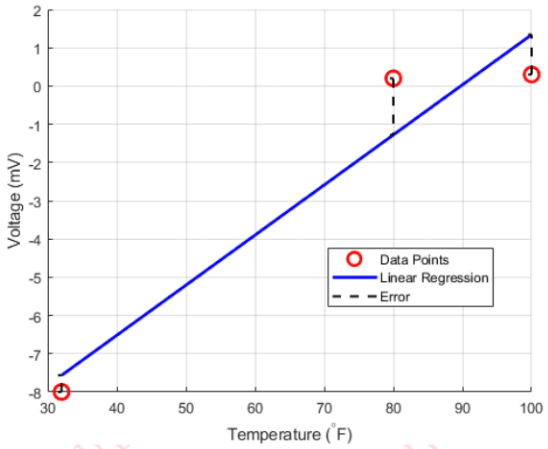


Figure 1: Linear Regression Fit. The red circle represents the original data, the blue line is the fitted linear line, and the dashed line shows the error from the fitted line to the original data.

How about a polynomial?

To find a non-linear function, we can enhance the original features by applying non-linear transformations, also known as kernels. For example, if we believe a second-order polynomial would be a better fit, we can add an extra feature like x^2. Table 2 provides an example of an augmented version of Table 1.

	Data 1	Data 2	Data 3
Temperature, x (°F)	32	80	100
Augmented Feature, x^2	32^2	80^2	100^2
Voltage, y (mV)	-8.	0.2	0.3

Table 2: Augmented Fake Measurement.

Now we aim to find a new line that

$$\underbrace{\begin{bmatrix} -8. \\ 0.2 \\ 0.3 \end{bmatrix}}_{y} = \underbrace{\begin{bmatrix} 32 & 32^2 & 1 \\ 80 & 80^2 & 1 \\ 100 & 100^2 & 1 \end{bmatrix}}_{X} \underbrace{\begin{bmatrix} m_1 \\ m_2 \\ b \end{bmatrix}}_{\theta}. \tag{6}$$

This approach is quite flexible. For example, we can use functions like $\sin(\omega \cdot x)$ and $\cos(\omega \cdot x)$ where ω is a predefined frequency (or another learnable parameter). The parameters obtained from fitting these sinusoidal functions correspond to the Fourier coefficients.

In fact, deep learning (neural networks) in modern AI is the most well-known example of this approach. For instance, consider an input image and an output label that classifies it as "hot-dog" or "not hot-dog." Here, the number of features equals the number of pixels in the image, and the goal is to find a non-linear function that minimizes error using gradient descent, which is a fundamental concept in most AI applications.

How much data do we need and what type of non-linear function should we use?

It's clear that having more data generally makes the fitted line better reflect the true relationship. For instance, large language models typically use all the data available on the internet. However, how much data is enough for a good fit is still an open question, with different opinions depending on the context. This ties into the concept of "goodness of fit" in statistical analysis.

Another key question is: what type of non-linear function should we use? This is also an ongoing area of research. One common strategy is to choose an overparameterized model, as it often provides better flexibility. For example, a second-order polynomial includes linear functions as a special case (when the quadratic coefficient is zero). Similarly, Taylor and Fourier series show that an infinite-order polynomial or sinusoidal model can approximate any function. Unfortunately, we can't use an infinite order in practice, so we can only make a finite truncation.

However, the drawback is that having more parameters means more computing power is needed to find the best solution. This creates a practical trade-off, where the decision depends on the specific problem you are trying to solve. Table 3 lists the number of parameters used by the most popular large language models.

Model Name	Number of Parameters	Company
GPT-3	175 billion	OpenAI/Microsoft
BERT	110 million	Google
GPT-2	1.5 billion	OpenAI/Microsoft
Transformer-XL	257 million	Google/CMU
T5	11 billion	Google
RoBERTa	355 million	Facebook AI/UW-Seattle
Llama	70 billion	Facebook AI
Claude	87 billion	Anthropic/Amazon
Gemini	92 billion	Google

Table 3: Number of Parameters in Popular Large Language Models

EE 2115 Lab 1

Sean Yin

Jan 27, 2025

University of Minnesota
Driven to Discover®

Goal: Add specific technical details from EE 2015.

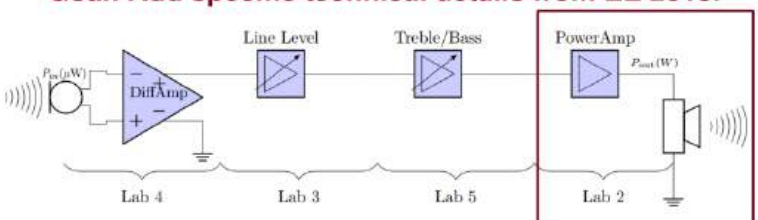


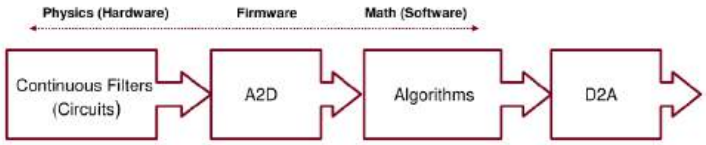
FIG. 1: Essential components of a hi-fi audio amplifier system to be implemented in EE 2015 Laboratory.

- **Lab 1: AMS + DSP.**

Lab 2, 3, 4

Workflow

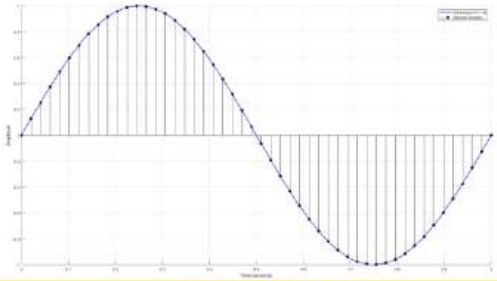
- Algorithms: My research area involves a lot of mathematical work, such as FFT (Fast Fourier Transform), machine learning, and more.



- Limit the original signal to specific working frequencies (and handle other tasks better suited for continuous time).
- The sampling frequency is determined by the Nyquist Criteria, which is twice the highest working frequency.
- Different interpolation methods are used to convert the processed discrete signal (vector) back into a continuous form.

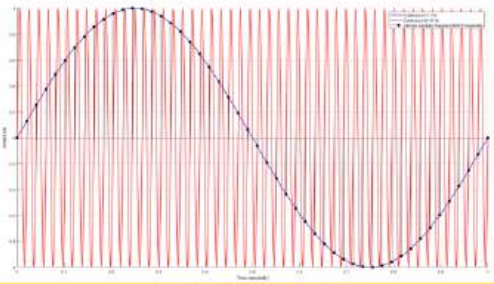
Discretization

- Sample the continuous time signal to be a discrete vector with some sampling rate.



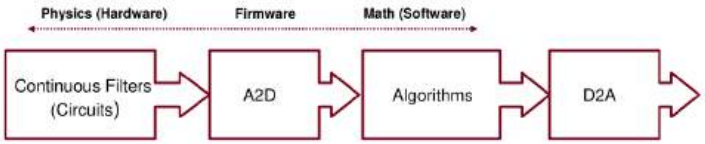
Issue: Aliasing

- Aliasing: the representation isn't unique, and we can't tell which are the original signals
- Solution: Use a continuous filter to limit the input signal to specific working frequencies.



Workflow

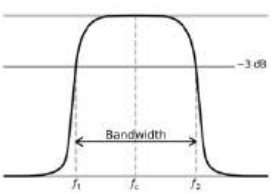
- Algorithms: My research area involves a lot of mathematical work, such as FFT (Fast Fourier Transform), machine learning, and more.

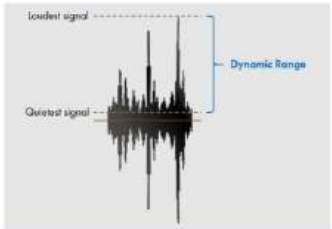


- Limit the original signal to specific working frequencies (and handle other tasks better suited for continuous time).
- The sampling frequency is determined by the Nyquist Criteria, which is twice the highest working frequency.
- Different interpolation methods are used to convert the processed discrete signal (vector) back into a continuous form.

Key Features

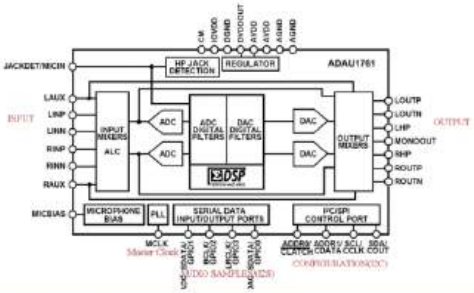
- Bandwidth.
- Dynamic Range.
- Number of bits per sample.
  - We only use floating-point numbers, not actual real numbers. We discretize both in terms of time and numerical values (quantization).





Lab 1

- Follow the SOP to play with the codec.





UNIVERSITY OF MINNESOTA

Driven to Discover®

Crookston Duluth Morris Rochester Twin Cities

The University of Minnesota is an equal opportunity educator and employer.

In Lab 1B, we will explore how programmable gain behaves by modifying the provided code, rather than using the variable analog gain controlled by circuits. This lab only requires **changing a single line of code**, so if you're not interested in the topic, you can skip to the last part of the document.

However, if you are interested, this exercise will(probably) help you understand the concept of mixed-signal systems, where part of the signal is controlled by analog circuits and part by programmable code. The choice between using hardware (circuits) or software (code) depends on the specific problem and design requirements.

Line-In vs Line-Out Gain

This section covers the difference between Line-In and Line-Out gain in the ADAU1761 audio codec, how to modify the gain levels, and how to implement the changes in C code.

Feature	**Line-In Gain (Recording)**	**Line-Out Gain (Playback)**
Purpose	Amplifies incoming audio before ADC conversion	Amplifies outgoing audio before DAC output
Registers	R5 (Left), R7 (Right)	R31 (Left), R32 (Right)
Affects	Recording sensitivity	Playback loudness
Changing Value	Boosts or reduces input signal	Boosts or reduces output volume
Risk of Distortion	If too high, input signal clips before ADC	If too high, output signal clips after DAC

Table 1: Comparison of Line-In Gain and Line-Out Gain

Where to Change the Gain in the Codec Code

The gain settings for the audio codec in your provided code are mainly found in the `AudioConfigureJacks()` and `LineinLineoutConfig()` functions. These functions configure recording mixers, ADCs, DACs, and playback mixers.

Key Places to Modify Gain

Recording Mixer Gain (Microphone / Line-In)

Adjusts the gain for input sources (microphone or line-in).

```
AudioWriteToReg(R5_RECORD_MIXER_LEFT_CONTROL_1, 0x07); // Gain for Left Line-In (6 dB)
AudioWriteToReg(R7_RECORD_MIXER_RIGHT_CONTROL_1, 0x07); // Gain for Right Line-In (6 dB)
```

Listing 1: Configuring Line-In Gain. Modify values (0x07 is 6 dB) for different gain.

Playback Mixer Gain (DAC to Output)

Adjusts the gain for output signals (headphones, speakers).

```
AudioWriteToReg(R26_PLAYBACK_LR_MIXER_LEFT_LINE_OUTPUT_CONTROL, 0x07); // Left output gain
    (6 dB)
AudioWriteToReg(R27_PLAYBACK_LR_MIXER_RIGHT_LINE_OUTPUT_CONTROL, 0x07); // Right output gain
     (6 dB)
```

Listing 2: Configuring Playback Gain. Modify values (0x07 is 6 dB) for different gain.

Line Output Volume Control (Final Output Gain)

Controls the final amplification to speakers or headphones.

```
AudioWriteToReg(R31_PLAYBACK_LINE_OUTPUT_LEFT_VOLUME_CONTROL, 0xFE); // Left channel output gain
AudioWriteToReg(R32_PLAYBACK_LINE_OUTPUT_RIGHT_VOLUME_CONTROL, 0xFE); // Right channel output gain
```

Listing 3: Configuring Line Output Gain. Modify values to adjust loudness (higher values increase gain).

Headphone Output Gain

If using headphones, modify:

```
AudioWriteToReg(R29_PLAYBACK_HEADPHONE_LEFT_VOLUME_CONTROL, 0xFF); // Mute left HP output
AudioWriteToReg(R30_PLAYBACK_HEADPHONE_RIGHT_VOLUME_CONTROL, 0xFF); // Mute right HP output
```

Listing 4: Configuring Headphone Gain. Change `0xFF` (mute) to a lower value to enable and set gain.

How to Increase the Output Volume to the Next Level

To increase the volume by one level, you need to increment the `LOUTVOL[5:0]` and `ROUTVOL[5:0]` values in `R31` and `R32` by 1.

Register Structure of R31 and R32

The `LOUTVOL[5:0]` and `ROUTVOL[5:0]` fields in `R31` and `R32` are bit[5:0] of the register values.
The current code sets:

```
AudioWriteToReg(R31_PLAYBACK_LINE_OUTPUT_LEFT_VOLUME_CONTROL, 0xFE);
AudioWriteToReg(R32_PLAYBACK_LINE_OUTPUT_RIGHT_VOLUME_CONTROL, 0xFE);
```

Listing 5: Current Volume Register Settings

- **Hex `0xFE` in binary:** `11111110` **Extracting bits [5:0]** → `111110` (Binary) = **Decimal 62**.

Register	Hex Value	Binary (8-bit)	L/ROUTVOL[5:0]	Decimal Value
R31 (LOUTVOL[5:0])	0xFE	11111110	111110	62
R32 (ROUTVOL[5:0])	0xFE	11111110	111110	62
After Increase	**0xFF**	**11111111**	**111111**	**63**

Table 2: Recorded Values of LOUTVOL[5:0] and ROUTVOL[5:0]

Since the current value is 62 (`0xFE`), increasing it to 63 (`0xFF`) will set the volume to its maximum level.

```
AudioWriteToReg(R31_PLAYBACK_LINE_OUTPUT_LEFT_VOLUME_CONTROL, 0xFF); // Increase Left Line-Out Volume to 63
AudioWriteToReg(R32_PLAYBACK_LINE_OUTPUT_RIGHT_VOLUME_CONTROL, 0xFF); // Increase Right Line-Out Volume to 63
```

Listing 6: Increasing Volume to Max

Before: `0xFE` (62 in decimal) **After:** `0xFF` (63 in decimal, MAX volume)

I2C vs. I2S

Both **I2C (Inter-Integrated Circuit)** and **I2S (Inter-IC Sound)** are communication protocols used in embedded systems. However, they serve different purposes:

- **I2C**: A serial communication protocol primarily used for **low-speed data transfer** between microcontrollers and peripherals (Each device has a 7-bit or 10-bit address).
- **I2S**: A serial bus protocol designed for **real-time digital audio transmission**.

Feature	Description
Purpose	I2C: Control and configure peripherals I2S: Transfer digital audio data
Speed	I2C: 100 kHz – 3.4 MHz I2S: 44.1 kHz – 192 kHz (Audio Sample Rate)
Data Type	I2C: Configuration data (e.g., gain settings) I2S: Real-time audio data
Communication Type	I2C: Master-Slave I2S: Typically one-way
Number of Wires	I2C: 2 (SCL, SDA) I2S: 3-4 (BCLK, WS, SD, MCLK)
Full-Duplex Support?	I2C: No I2S: Yes (can handle stereo/multi-channel audio)
In ADAU1761	I2C: Setting up registers (gain, sampling rate) I2S: Sending/receiving digital audio

Table 3: Comparison Between I2C and I2S. In Number of Wires: **Bit Clock (BCLK)** – Defines data transmission timing. **Word Select (WS) / Left-Right Clock (LRCLK)** – Identifies left/right channel data. **Serial Data (SD)** – Transmits actual audio data. **(Optional) Master Clock (MCLK)** – Provides system clock.

Code Examples

```c
void AudioWriteToReg(unsigned char u8RegAddr, unsigned char u8Data) {
    unsigned char u8TxData[3];
    u8TxData[0] = 0x40;  // Device address
    u8TxData[1] = u8RegAddr;
    u8TxData[2] = u8Data;
    XIicPs_MasterSendPolled(&Iic, u8TxData, 3, (IIC_SLAVE_ADDR >> 1));
    while (XIicPs_BusIsBusy(&Iic));
}
```

Listing 7: Writing to ADAU1761 Register using I2C

```c
void audio_stream(){
    u32 in_left, in_right;
    while (!XUartPs_IsReceiveData(UART_BASEADDR)){
        in_left = Xil_In32(I2S_DATA_RX_L_REG);  // Read left channel
        in_right = Xil_In32(I2S_DATA_RX_R_REG); // Read right channel
        Xil_Out32(I2S_DATA_TX_L_REG, in_left);  // Write to left output
        Xil_Out32(I2S_DATA_TX_R_REG, in_right); // Write to right output
    }
}
```

Listing 8: Streaming Audio Using I2S

Schedule and Policy

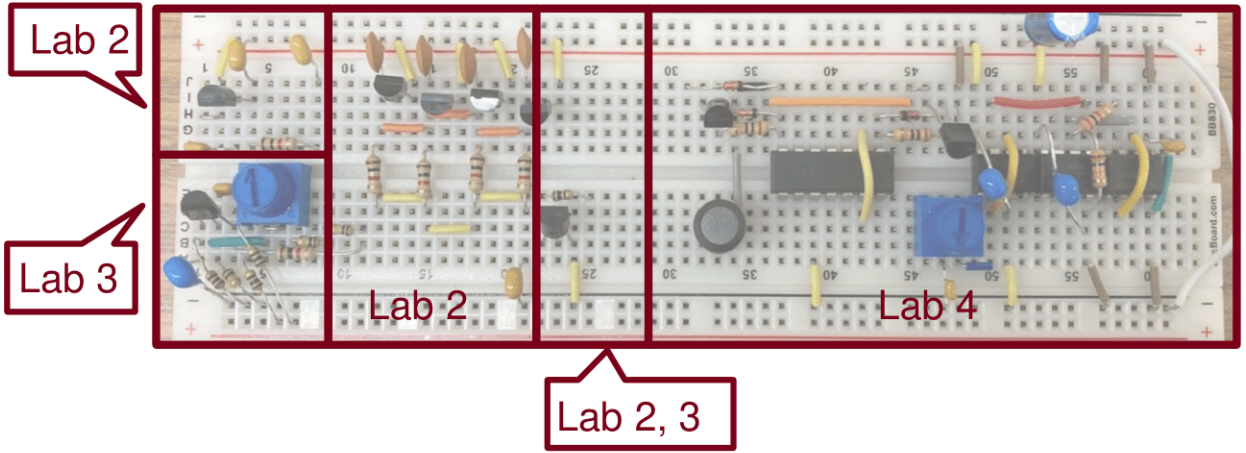
- **Lab Schedule**
  - **Lab 1:** 1/27, 2/3 (No prelab, Notebook due: 2/10).
  - **Lab 2:** 2/10, 2/17 (Prelab:2/17, Notebook due: 2/24).
  - **Lab 3:** 2/24, 3/3 (Prelab:3/3, Notebook due 3/17).
  - **Lab 4:** 3/17, 3/24, 3/31 (No prelab, Report due 4/7).
- **Grading:**
  - **Pre-lab:** Due before your lab class.
  - **Lab Notebook/Report:** Due before the next lab class.
  - **Lab Grade:** 50% demonstration to TA, 50% notebook/report. More details will be sent later.
- **Office Hours:**
  - **Time:** After Lab Period on Mondays.
  - You can attend any TA's office hours. If a different TA checks you off, email me and CC them.
  - You can also complete the lab on your own and send me photos as proof.

Labs Overview

- **Lab 1**
  - Introduces an Analog Mixed Signal (AMS) system for processing audio signals.
    - AMS: Combines analog and digital signal processing in one system.
  - ADAU1761 audio Codec chip mounted on the ZedBoard™ for audio processing.
- **Lab 2**
  - Test a MOSFET with varying voltages to create characterization curves.
  - Analyze a single-transistor inverter and multiple inverters to link performance and delay measurements to the curves.
- **Lab 3**
  - Evaluate a MOSFET, test a single-transistor amplifier's gain and bandwidth, and compare actual to theoretical performance using key parameters.
- **Lab 4**
  - Explore Class D Audio Amplifier stages.
    - Set up and assess the PWM input stage and recover audio signals using a lowpass filter.
    - Focus on the power amplifier stage to enhance efficiency and reduce heat waste, referencing the LM386 amplifier studied in EE 2015.
    - Investigate the PA output stage with MOSFET transistors and the IR2110 .

Lab Components (Some from EE 2015)

- Lab 2
  - 2N7000 MOSFET (4), 106 Capacitor (2), 103 Capacitor (4), 104 Capacitor (1).
- Lab 3
  - 2N7000 MOSFET (1), 106 Capacitor (1), 104 Capacitor (1), 104 Potential meter (1).
- Lab 4
  - 2N7000 MOSFET (2), 102 Capacitor (2), 106 Capacitor (1), 476 Capacitor (1), 103 Potential meter (1), LM555 (1), LM393 (1), LM741 (1), IR2110 (1), Inductor (1).



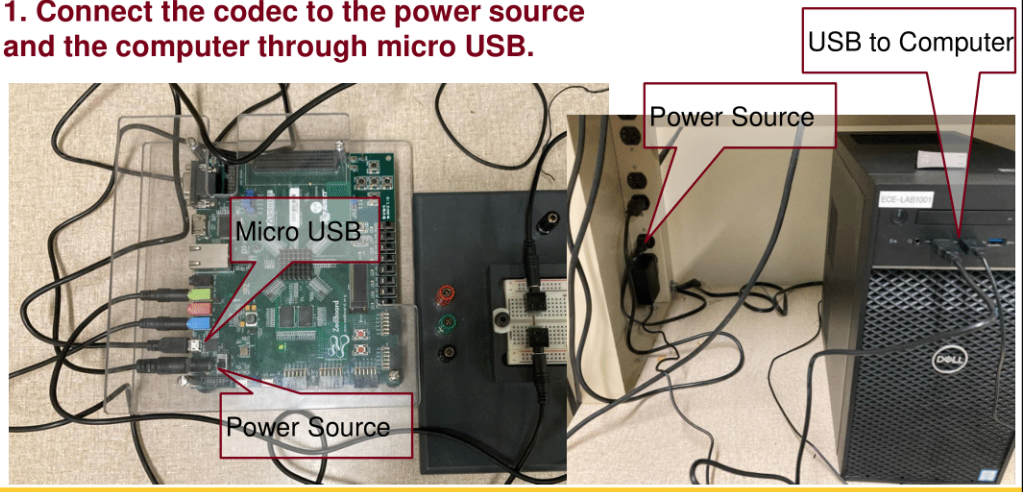
EE 2115 Lab 1 SOP

Sean Yin

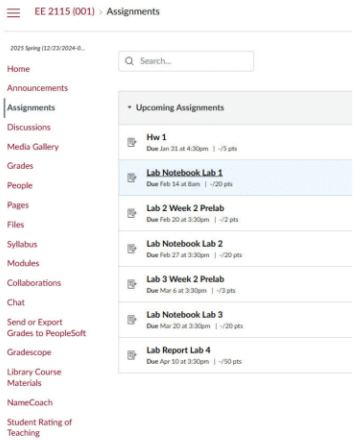
Jan 27, 2025

UNIVERSITY OF MINNESOTA
Driven to Discover®

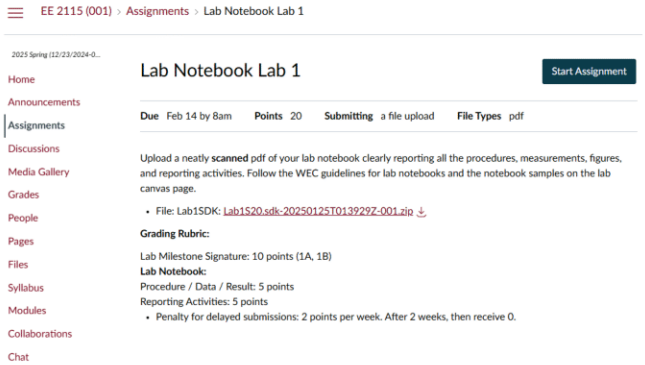
1. Connect the codec to the power source and the computer through micro USB.



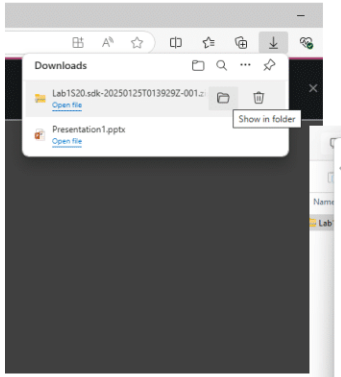
2. Assignment >> Lab Notebook Lab 1.



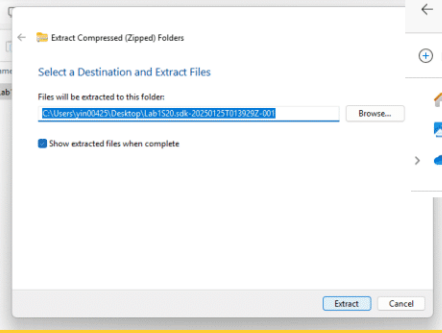
3. Download SDK.



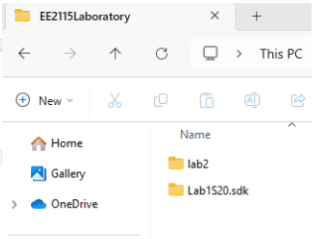
4. Show it in the folder.



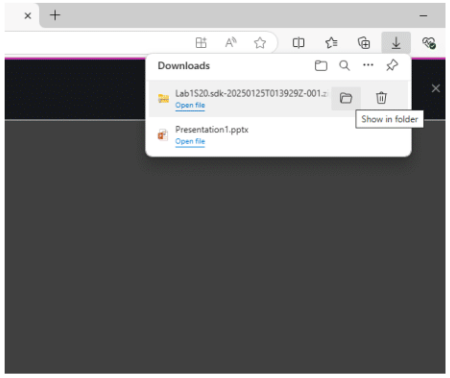
5. Unzip.



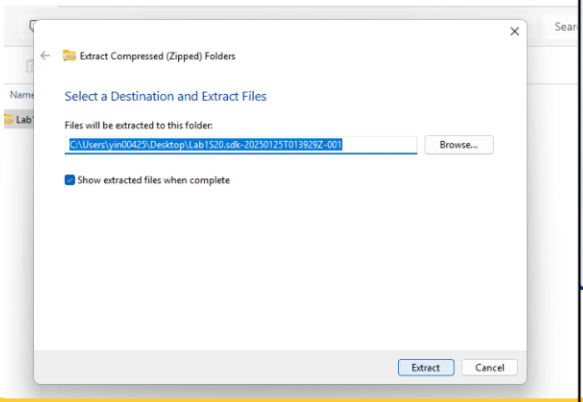
6. Find Lab1S20.sdk folder.



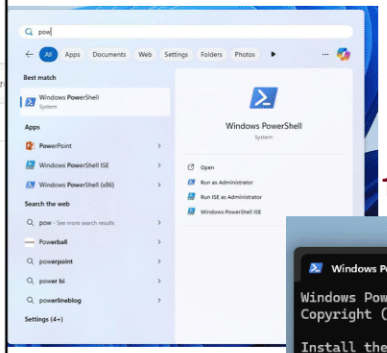
7. Show it in the folder.



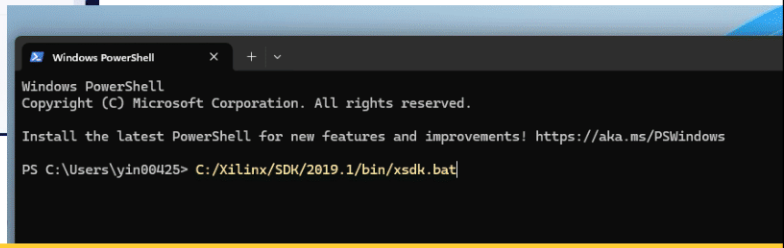
8. Unzip.



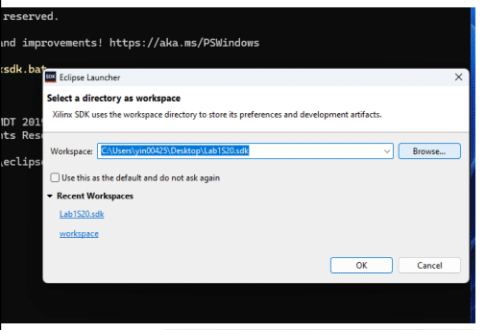
9. Find the terminal.



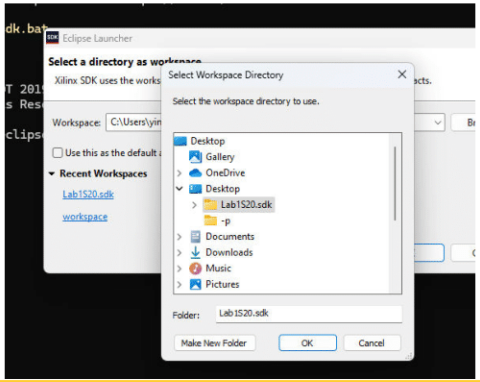
10. Type “C:/Xilinx/SDK/2019.1/xsdk.bat”>>enter.



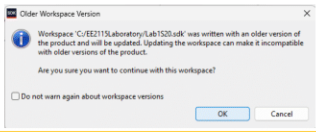
11. Browse.



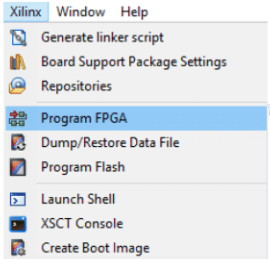
12. Find the folder you saved >> OK.



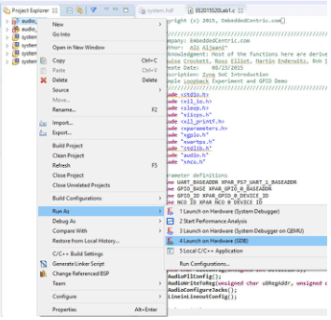
12.5. OK.



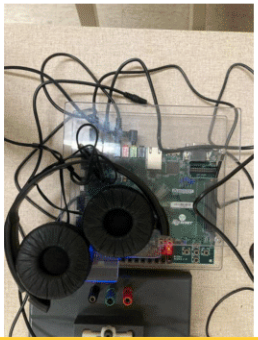
13. Xilinx >> Program FPGA.



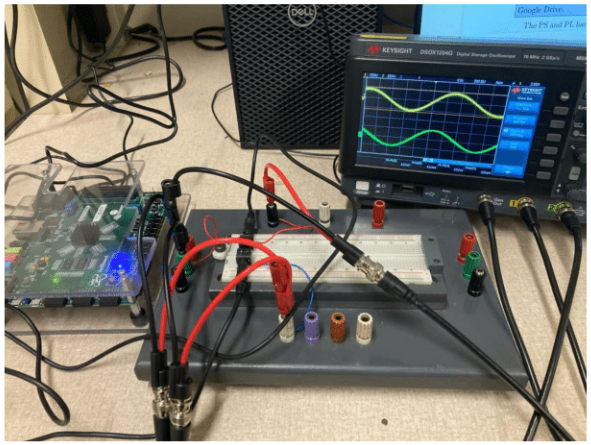
14. Right Click audio_test >> Run As>> 4 Launch……



15. Listen to different sin waves.



16. Follow Manuel for standard FRA analysis.



UNIVERSITY OF MINNESOTA

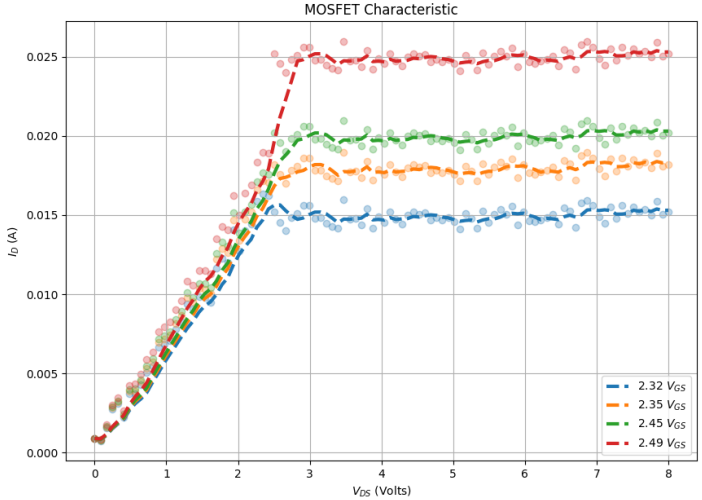
Driven to Discover®

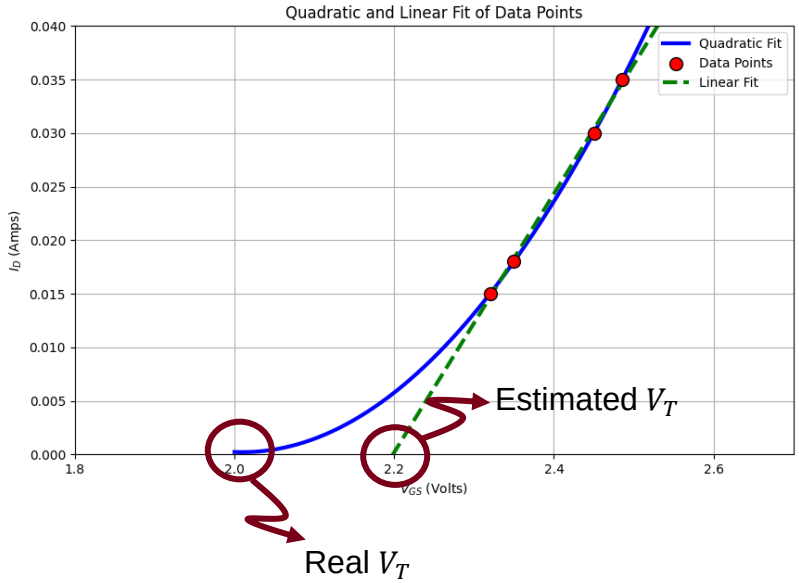
Crookston Duluth Morris Rochester Twin Cities

The University of Minnesota is an equal opportunity educator and employer.

Find Threshold Voltage

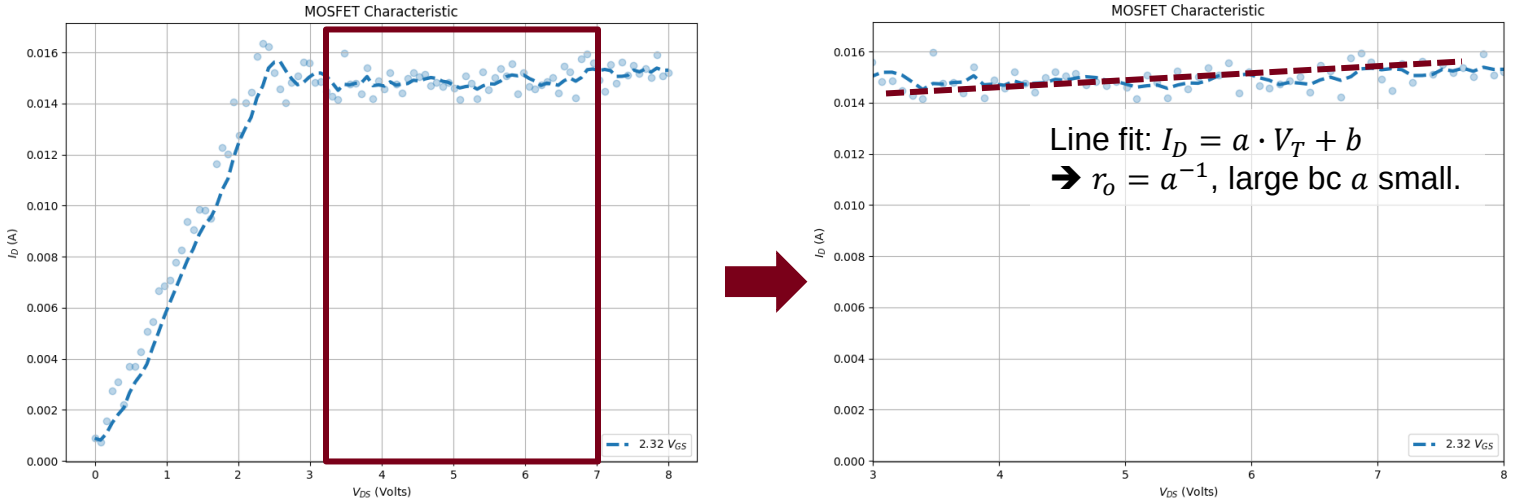
- Focus on the saturation region—each V_{GS} gives you one I_D.
- Fit a function to find the threshold voltage (V_T).





Find r_o

- Focus on the saturation region for one V_{GS}.
- Fit a function to find the $r_o = a^{-1}$.
- r_o should be large because (1) The saturation region should be nearly horizontal. (2) The parallel r_o should be large enough so that most of I_D does not flow through it.



EE 2115 Lab 4 Handout

Sean Yin

March 17, 2025



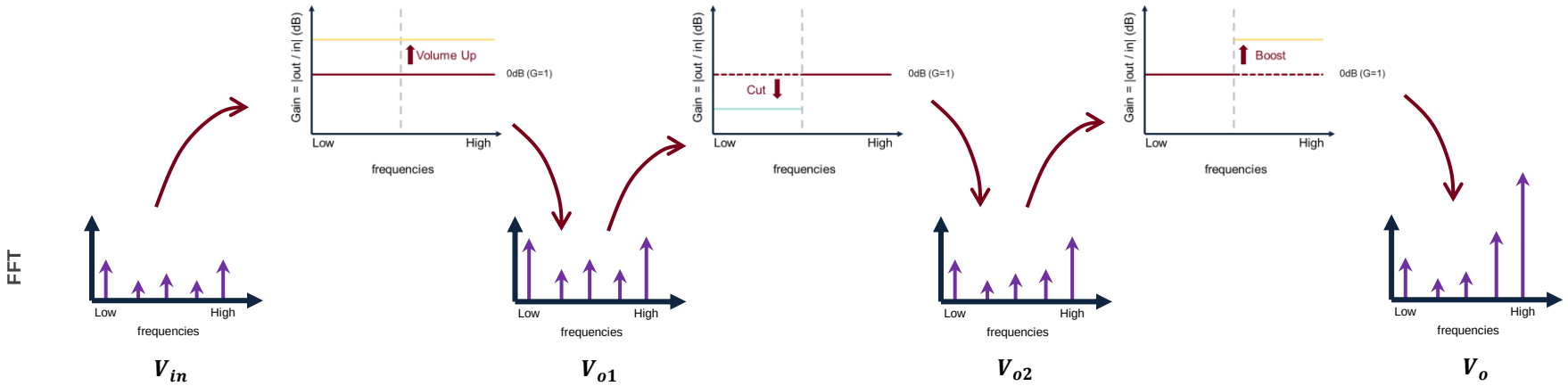
UNIVERSITY OF MINNESOTA
Driven to Discover®

EE 2015 Audio Full-Chain

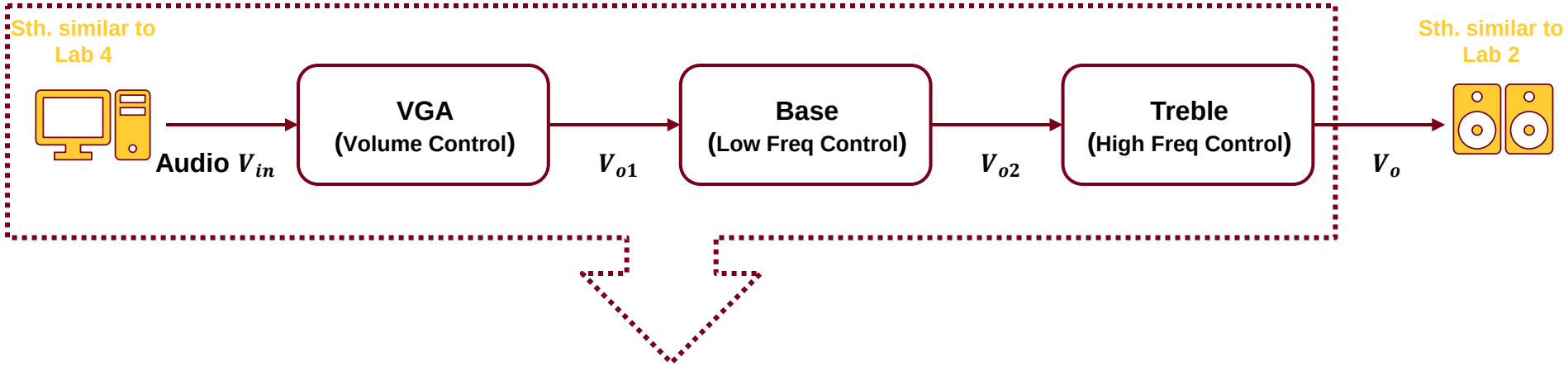
Sth. similar to Lab 4

Sth. similar to Lab 2



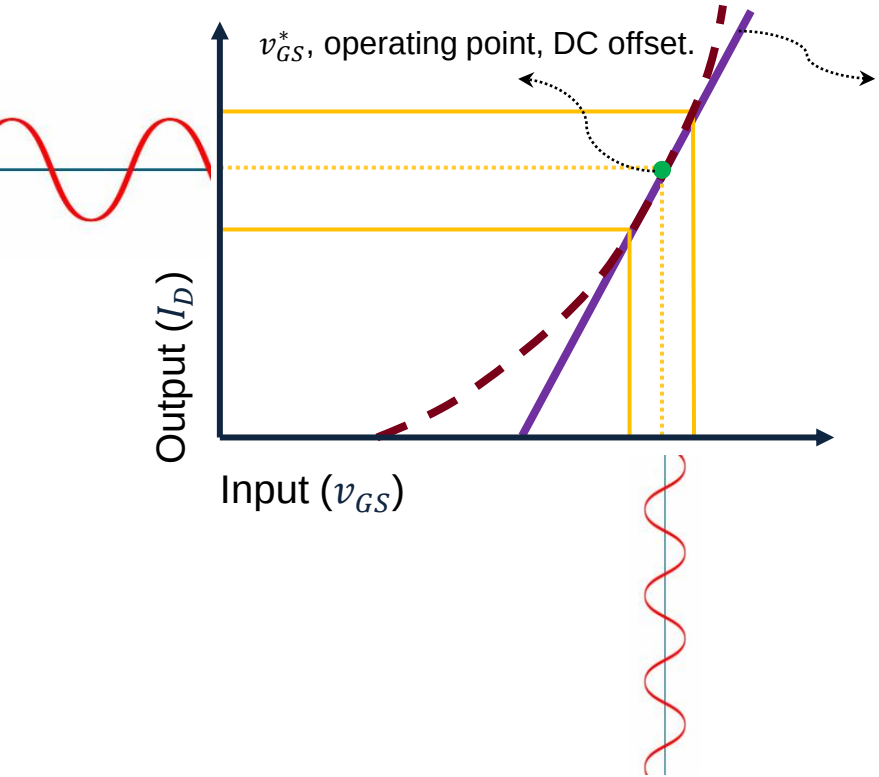


EE 2015 Audio Full-Chain: Issues



- The signal comes from converting a digital (discrete) computer signal back into an analog (continuous) signal (D2A) ➔ Small Signal.
- Issue 1: The signal amplitude might be smaller than the noise from wires/cables.
  ➔ Solution: Use a differential amplifier (similar to Lab 4 in EE 2015).
- Issue 2: The signal is too small to drive a speaker.
  ➔ Solution: Use a power amplifier (the main focus of this lab).

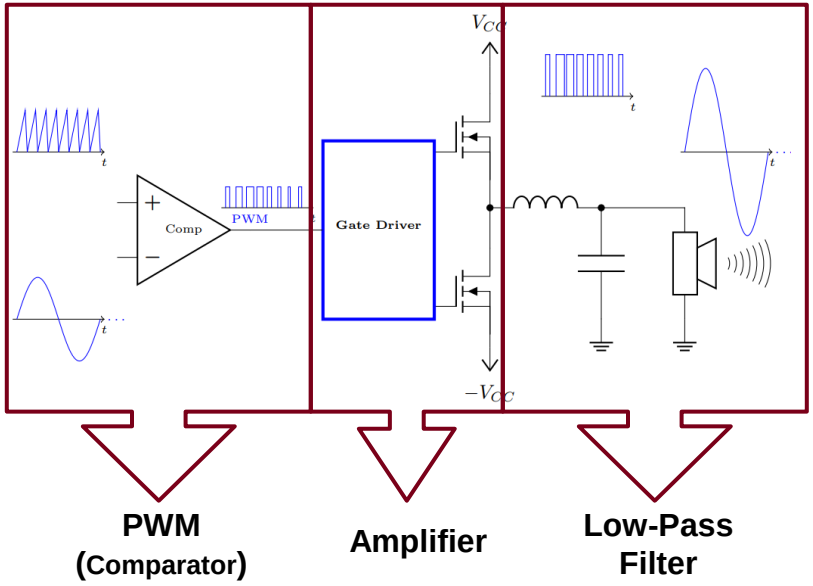
EE 2115 Lab 3: Class A Power Amplifier

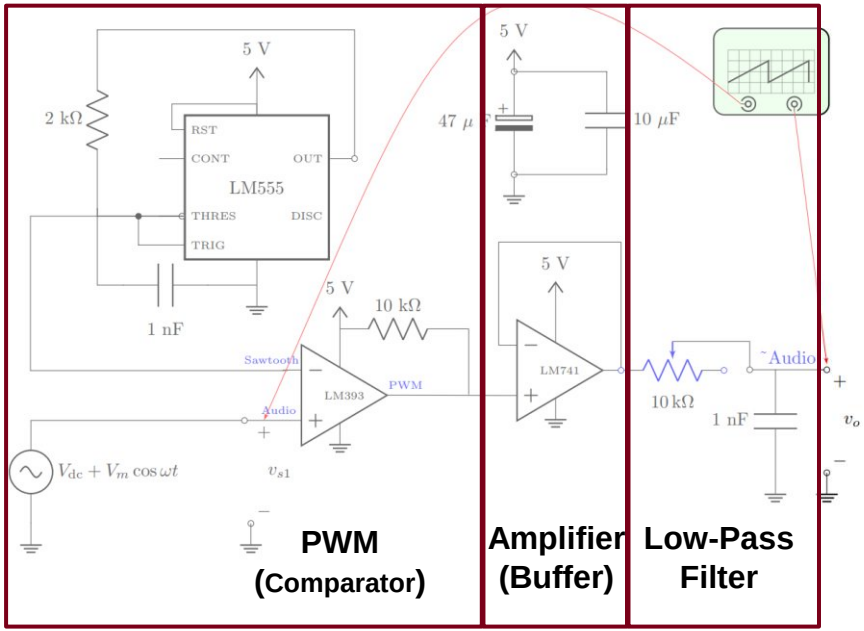


- $I_D(v_{GS}) = K\,(1 + \lambda V_{DS})(v_{GS} - V_T)^2$
- $I_D(\mathbf{v}_{GS,AC}) = I_D(v_{GS}^*) + g_m(v_{GS}^*) \cdot \mathbf{v}_{GS,AC}$.
- Issue:
  - Constant current ($I_D(v_{GS}^*)$) ➔ constant power ➔ constant heat ➔ damage the device.
- Solution: Only input signal exists, turn on the amplifier. (eliminate the constant current)
  - Class B (never used, high distortion),
  - Class AB (combining Class A and Class B).
  - 50 % Efficiencies.
- Another solution: Class D (main focus).
  - 90 % Efficiencies.
  - Widely used in subwoofers and Bluetooth speakers because it generates very little heat and can be used in small devices.

EE 2115 Lab 4: Class D Power Amplifier.

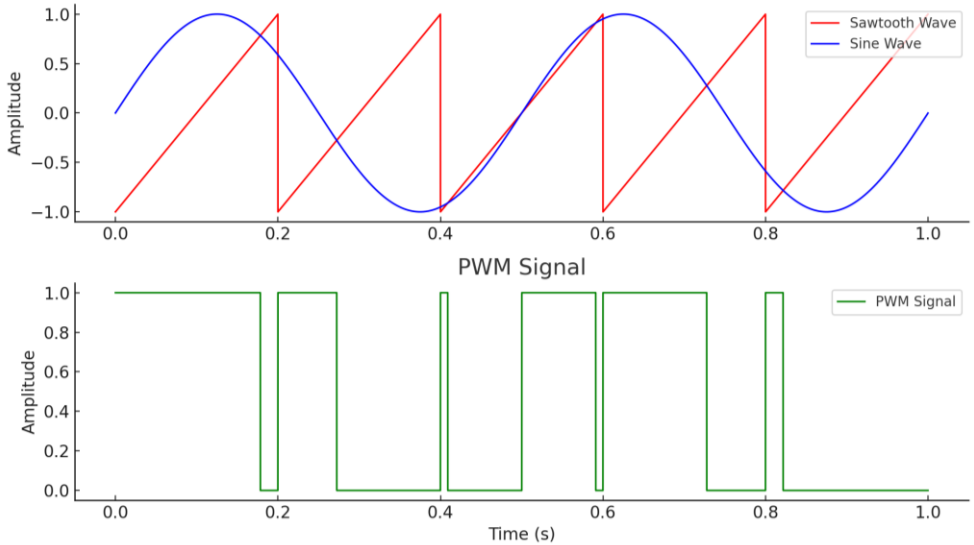






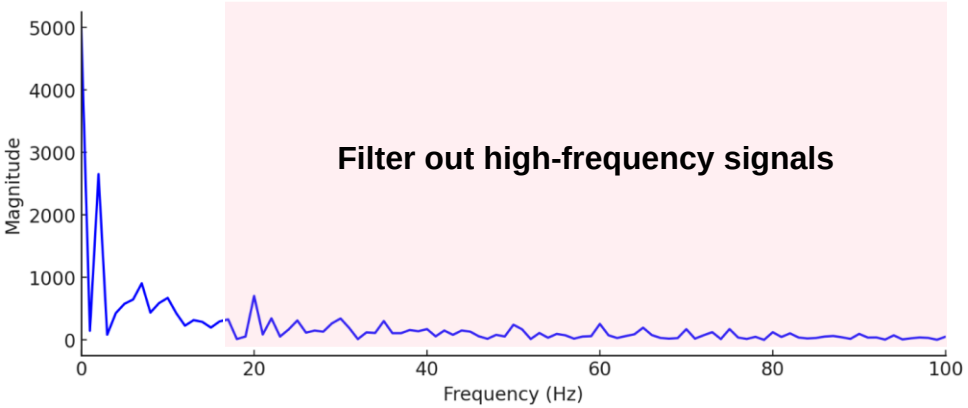
Pulse Width Modulation (PWM)

- Comparator: Compares the input signal (sine wave) with a high-frequency sawtooth wave.
  - If the input signal is greater than the sawtooth wave, the output is 1; otherwise, it is 0.
  - pwm_signal = sine_wave > sawtooth_wave.



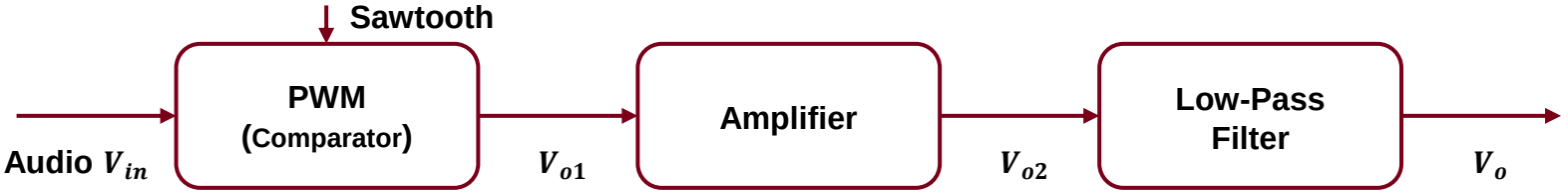
Low-pass Filter (Demodulation)

- Issue: The amplified PWM signal is not a recognizable audio signal for humans.
- Frequency Analysis of the PWM Signal.



- Solution: A **low-pass filter** removes unwanted high-frequency signals in PWM and helps recover the audio signal (sine wave).

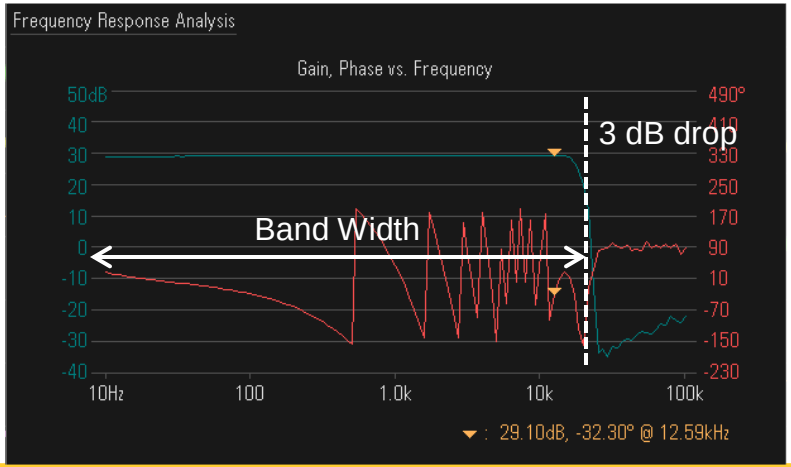
EE 2115 Lab 4: Class D Power Amplifier.

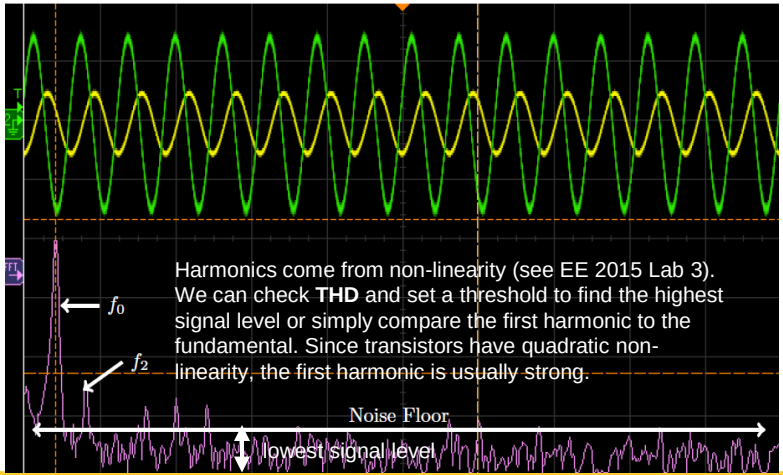


- PWM stage: A **comparator** compares the **input signal** with a **sawtooth wave**.
  - In Lab 4, the LM 555 generates the sawtooth signal, and the LM 393 is used as the comparator.
- The **resulting PWM** signal is then **amplified**.
  - In 4.A and 4.B, we use a simple op-amp (LM 741) as a **buffer**.
  - In 4.C, we first use IR2110 as a gate driver, then use **two transistors** for amplification—one for the positive half and one for the negative half of the signal.
  - Since the input itself is just a switching signal, the transistor amplifier also acts as a switch, **improving efficiency and reducing heat** (no constant current).
- Finally, the amplified PWM signal passes through a **low-pass filter** to **recover** the **amplified audio signal**.

Band Width and Dynamic Range

- Bandwidth: The range of frequencies a system can handle effectively.
- Dynamic Range: The ratio between the highest and lowest signal levels a system can handle.
  - The highest signal level: The maximum amplitude that keeps the system within the linear region, avoiding distortion or clipping.
  - The lowest signal level: The minimum detectable signal above the noise floor.







UNIVERSITY OF MINNESOTA

Driven to Discover®

Crookston Duluth Morris Rochester Twin Cities

The University of Minnesota is an equal opportunity educator and employer.